Kombinatorische Logik

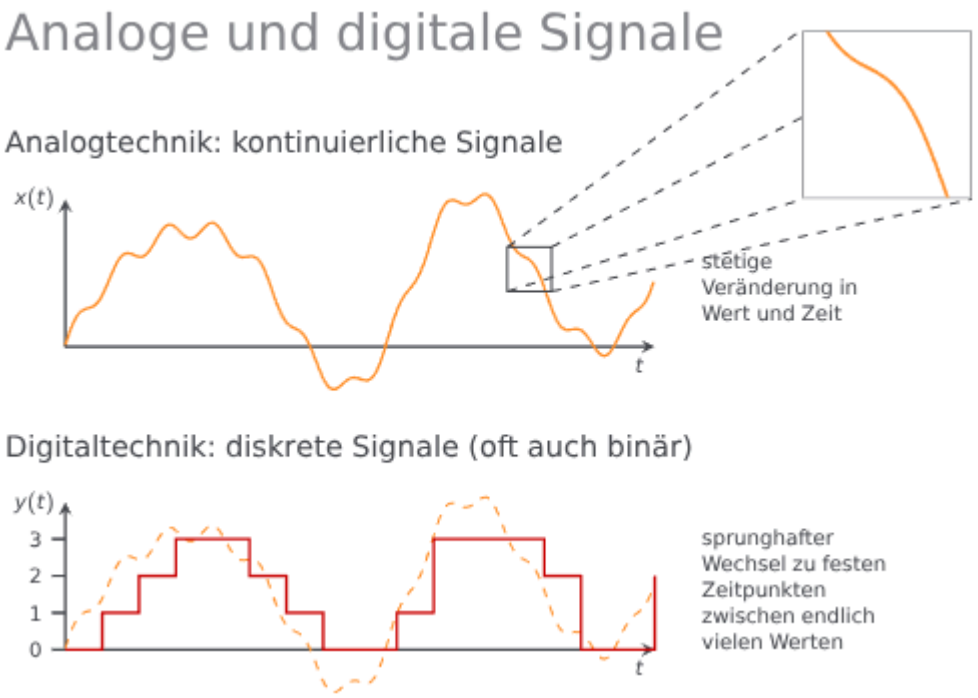
- Digitalrechner|Vor-/Nachteile Analogrechner | Digitalrechner
- Kombinatorische Logik
- Definition
- Elementare Operatoren
- Boolesche Axiome
- Sätze
- Weitere Eigenschaften einer Booleschen Algebra
- De Morgansche Gesetze
- Einstellige Boolesche Funktionen
- Zweistellige Boolesche Funktionen
  - Wichtige Bezeichnungen der zweistelligen Booleschen Funktionen
- Gatter
- Entwurfskriterien
  - Laufzeitverhalten
- Wahrheitstabelle → Boolescher Ausdruck (KDNF und KKNF)
- Bemerkung: Technische Realisierung
- Resolutionsregeln
- Karnaugh-Veitch-Diagramme (KV-Diagramme)
- Dekodierer
- Kodierer
  - Prioritätskodierer
- Multiplexer
  - Demultiplexer

Einleitung

- Ziel: Entwurf digitaler logischer Schaltungen (UND, ODER, NICHT usw.)
- Kombinatorische Logik: Die Ausgänge hängen **nur von den aktuellen Eingängen** ab, es gibt keinen Speicher. Das Gegenstück ist die Sequenzielle Logik.

---

1. Grundlagen der Digitaltechnik



Ein analoges Signal hat unendlich viele mögliche Werte. Die Digitaltechnik **diskretisiert** es, d. h. sie beschränkt die Werte auf endlich viele (z. B. $0, 1, 10, 11, \ldots$).

- Binäre Werte können auch als **0/1, L/H (Low/High) oder F/T (falsch/wahr)** dargestellt werden.
- **Alle Arten von Daten werden digital als diskrete Zahlen gespeichert** (z. B. Text als ASCII, Bilder als Pixelwerte).

Vor-/Nachteile Analogrechner | Digitalrechner

Analogrechner rechnen mit kontinuierlichen Signalen (z. B. Spannungen), Digitalrechner nur mit 0 und 1.

	Analogrechner	Digitalrechner
Signal	kontinuierlich	diskret (0/1)
Genauigkeit	begrenzt durch Rauschen und Bauteiltoleranzen	beliebig steigerbar (mehr Bits)
Störanfälligkeit	hoch	gering (nur zwei Pegel müssen unterschieden werden)
Speichern/Kopieren	verlustbehaftet	verlustfrei
Flexibilität	feste Schaltung pro Aufgabe	frei programmierbar

Kombinatorische Logik

- Digitale Zustände werden durch **Gatter** miteinander verknüpft, sodass sich aus den Eingangszuständen ein Ausgangszustand ergibt.
- Mehrstellige digitale Signale werden durch **Vektoren** dargestellt, z. B. $x = (x_2, x_1, x_0)$.

2. Boolesche Algebra

Definition

Die Kombination der Booleschen Menge $B = 0, 1$ mit den Operatoren **OR, AND und NOT** wird als **Boolesche Algebra** bezeichnet.

Schreibweisen

- $(\mathbb{B}, \text{AND}, \text{OR}, \text{NOT})$
- $(\mathbb{B}, \cdot, +, -) \quad = \quad (\{0,1\}, \cdot, +, -)$
- $\mathbb{B}(\wedge, \vee, \neg) \quad$ auch $\quad B(\wedge, \vee, \neg)$

Elementare Operatoren

OR-Operator

logische Summe (ODER)

$+ \quad \vee$

x_1	x_2	x_1 OR x_2
0	0	0
0	1	1
1	0	1
1	1	1

Das Ergebnis ist 1, falls **mindestens ein** Operand den Wert 1 annimmt.

AND-Operator

logisches Produkt (UND)

$\cdot \quad * \quad \wedge$

x_1	x_2	x_1 AND x_2
0	0	0
0	1	0
1	0	0
1	1	1

Das Ergebnis ist 1, genau dann wenn **beide** Operanden den Wert 1 annehmen.

NOT-Operator

Invertierung (NICHT)

$\overline{x} \quad -x \quad \neg$

x	NOT x
0	1
1	0

Das Ergebnis ist 1, genau dann wenn der Operand den Wert 0 annimmt.

Operator	Schreibweise	Ergebnis
OR	$a + b$	1, wenn **mindestens ein** Eingang 1 ist
AND	$a \cdot b$	1, wenn **alle** Eingänge 1 sind
NOT	$\overline{a}$	invertiert den Eingang

Boolesche Axiome

Axiome

der Booleschen Algebra zur Umformung logischer Gleichungen

Kommutativität

$$x_1 + x_2 = x_2 + x_1 \tag{1}$$
$$x_1 \cdot x_2 = x_2 \cdot x_1 \tag{2}$$

Distributivität

$$x_1 \cdot (x_2 + x_3) = (x_1 \cdot x_2) + (x_1 \cdot x_3) \tag{3}$$
$$x_1 + (x_2 \cdot x_3) = (x_1 + x_2) \cdot (x_1 + x_3) \tag{4}$$

Neutrale Elemente

$$0 + x = x \tag{5}$$
$$1 \cdot x = x \tag{6}$$

Komplementäres Element

$$x + \overline{x} = 1 \tag{7}$$
$$x \cdot \overline{x} = 0 \tag{8}$$

Sätze

Sätze

abgeleitet aus den Axiomen

Idempotenz

$$x + x = x \tag{9}$$
$$x \cdot x = x \tag{10}$$

Assoziativität

$$x_1 + (x_2 + x_3) = (x_1 + x_2) + x_3 \tag{11}$$
$$x_1 \cdot (x_2 \cdot x_3) = (x_1 \cdot x_2) \cdot x_3 \tag{12}$$

Absorption

$$x_1 + (x_1 \cdot x_2) = x_1 \tag{13}$$
$$x_1 \cdot (x_1 + x_2) = x_1 \tag{14}$$

Substitution

$$x + 1 = 1 \tag{15}$$
$$x \cdot 0 = 0 \tag{16}$$

Doppelnegation

$$\overline{\overline{x}} = x \tag{17}$$

Weitere Eigenschaften einer Booleschen Algebra

- Komplement
  - $\overline{0} = 1$ und $\overline{1} = 0$
- Abgeschlossenheit
  - Boolesche Operationen liefern nur boolesche Werte als Ergebnis.
- Dualität
  - Für jede aus den Axiomen ableitbare Aussage existiert eine duale Aussage. Man erhält sie durch Vertauschen der Operatoren $+$ und $\cdot$ sowie der Werte 0 und 1.

De Morgansche Gesetze

Wichtig für die Vereinfachung von Formeln und für die Normalformen.

Sie beschreiben, wie man einen **gesamten Ausdruck negiert**: Die Negation wird "hineingezogen", dabei wechselt der Operator (AND ↔ OR).

1. $\overline{a \cdot b} = \overline{a} + \overline{b}$
2. $\overline{a + b} = \overline{a} \cdot \overline{b}$

Beispiel:

$$\overline{(x_1 + x_2) \cdot \overline{x_3}} = (\overline{x_1} \cdot \overline{x_2}) + x_3$$

---

3. Boolesche Funktionen

Eine Schaltfunktion mit $m = 1$ Ausgang heißt **n-stellige Boolesche Funktion**, mit n Eingängen und einem Ausgang.

- Jede Belegung der Eingänge kann auf 0 oder 1 abgebildet werden. Eine Wahrheitstabelle hat 2^n Zeilen, also gibt es 2^{2^n} verschiedene Boolesche Funktionen mit n Eingängen.
- Beispiele: $n = 1 \Rightarrow 2^2 = 4$ Funktionen, $n = 2 \Rightarrow 2^4 = 16$ Funktionen.

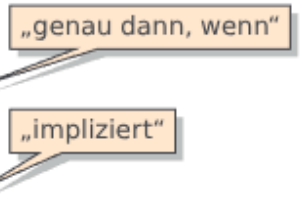
Einstellige Boolesche Funktionen

$f_0(x) = 0$ Kontradiktion (0-stellig)
$f_1(x) = x$ Identität
$f_2(x) = -x$ Negation
$f_3(x) = 1$ Tautologie (0-stellig)

Zweistellige Boolesche Funktionen

$x_2 =$ 0 1 0 1 $x_1 =$ 0 0 1 1		Term	Bezeichnung	Sprechweise
f_0	0 0 0 0	0	Nullfunktion	
f_1	0 0 0 1	$x_1 x_2$	Konjunktion	x_1 AND x_2
f_2	0 0 1 0	$x_1 \overline{x_2}$	1. Differenz	x_1 AND NOT x_2
f_3	0 0 1 1	x_1	1. Identität	
f_4	0 1 0 0	$\overline{x_1} x_2$	2. Differenz	NOT x_1 AND x_2
f_5	0 1 0 1	x_2	2. Identität	
f_6	0 1 1 0	$\overline{x_1} x_2 + x_1 \overline{x_2}$	Antivalenz	x_1 XOR x_2
f_7	0 1 1 1	$x_1 + x_2$	Disjunktion	x_1 OR x_2

$x_2 =$ 0 1 0 1 $x_1 =$ 0 0 1 1		Term	Bezeichnung	Sprechweise
f_8	1 0 0 0	$\overline{x_1 + x_2}$	Negatdisjunktion	x_1 NOR x_2
f_9	1 0 0 1	$(\overline{x_1} + x_2)(x_1 + \overline{x_2})$	Äquivalenz	$x_1 \Leftrightarrow x_2$
f_{10}	1 0 1 0	$\overline{x_2}$	2. Negation	NOT x_2
f_{11}	1 0 1 1	$x_1 + \overline{x_2}$	2. Implikation	$x_2 \Rightarrow x_1$
f_{12}	1 1 0 0	$\overline{x_1}$	1. Negation	NOT x_1
f_{13}	1 1 0 1	$\overline{x_1} + x_2$	1. Implikation	$x_1 \Rightarrow x_2$
f_{14}	1 1 1 0	$\overline{x_1 x_2}$	Negatkonjunktion	x_1 NAND x_2
f_{15}	1 1 1 1	1	Einsfunktion	



Wichtige Bezeichnungen der zweistelligen Booleschen Funktionen

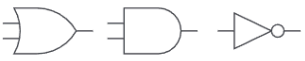
Bezeichnung	Gatter / Bedeutung	Formel
Konjunktion	AND – Gatter	$a \cdot b$
Disjunktion	OR – Gatter	$a + b$
Antivalenz	XOR – Gatter (genau eine Eingabe ist 1)	$a \oplus b$
Äquivalenz	XNOR ("genau dann, wenn")	$\overline{a \oplus b}$
Negatdisjunktion	NOR – Gatter	$\overline{a + b}$
Negatkonjunktion	NAND – Gatter	$\overline{a \cdot b}$
Implikation	"wenn a, dann b"	$a \Rightarrow b = \overline{a} + b$
Negation	NOT – Gatter (einstellig)	$\overline{a}$

4. Realisierung in Schaltungen

Die wichtigsten Booleschen Funktionen werden als eigene Gatter realisiert.

Gatter

Wichtige Boolesche Funktionen als Gatter

Eingabe		Ausgabe					
		elementar			kombiniert		
x_1	x_2	$x_1 + x_2$	$x_1 x_2$	$\overline{x_1}$	$\overline{x_1}x_2 + x_1\overline{x_2}$	$\overline{x_1 + x_2}$	$\overline{x_1 x_2}$
0	0	0	0	1	0	1	1
0	1	1	0	1	1	0	1
1	0	1	0	0	1	0	1
1	1	1	1	0	0	0	0
Schaltsymbol							
Bezeichnung		OR	AND	NOT	XOR	NOR	NAND

Siehe auch: Wichtige Bezeichnungen der zweistelligen Booleschen Funktionen

Entwurfskriterien

- **Flächenbedarf** → Anzahl der Gatter → Produktionskosten
- **Energieverbrauch**
- **Schaltgeschwindigkeit** → Anzahl der Schaltebenen
- **Laufzeitverhalten**

Laufzeitverhalten

Jedes Gatter braucht eine gewisse Zeit, um zwischen den beiden Zuständen umzuschalten (**Laufzeiteffekt**). Diese Zeit wird mit τ bezeichnet (wird im Kapitel Sequenzielle Logik wieder wichtig).

Durch unterschiedliche Laufzeiten auf verschiedenen Signalwegen kann es zu einer **Race Condition** kommen. Dadurch entstehen **Hazards**: ein unerwünschter Wert (Glitch) tritt am Ausgang kurzzeitig auf.

---

5. Normalformen (Kanonische Darstellungen)

Boolesche Funktionen lassen sich aus Produkt- und Summentermen aufbauen:

- **Produktterm** → Konjunktion von Variablen (UND)
- **Summenterm** → Disjunktion von Variablen (ODER)

Daraus ergeben sich:

- **DNF** (Disjunktive Normalform): Disjunktion von Produkttermen
- **KNF** (Konjunktive Normalform): Konjunktion von Summentermen

Eine eindeutige Darstellung einer Booleschen Funktion f als

- **Disjunktion von Mintermen** (Produktterme, die *alle* Variablen enthalten) heißt **KDNF** (Kanonische DNF),
- **Konjunktion von Maxtermen** (Summenterme, die *alle* Variablen enthalten) heißt **KKNF** (Kanonische KNF).

Wahrheitstabelle → Boolescher Ausdruck (KDNF und KKNF)

Zu jeder Booleschen Funktion gibt es genau eine KDNF und genau eine KKNF. Jede KDNF kann in eine KKNF umgewandelt werden und umgekehrt.

KDNF

1. Alle Zeilen betrachten, in denen der **Ausgang = 1** ist.
2. Pro Zeile einen Minterm (UND) bilden. Eingänge mit Wert **0** werden **negiert**.
3. Alle Minterme mit ODER verknüpfen.

KKNF

1. Alle Zeilen betrachten, in denen der **Ausgang = 0** ist.
2. Pro Zeile einen Maxterm (ODER) bilden. Eingänge mit Wert **1** werden **negiert**.
3. Alle Maxterme mit UND verknüpfen.

> **Beispiel XOR ($y = 1$ bei $(0,1)$ und $(1,0)$)**
>
> - KDNF: $y = \overline{a}, b + a, \overline{b}$
> - KKNF: $y = (a + b)(\overline{a} + \overline{b})$

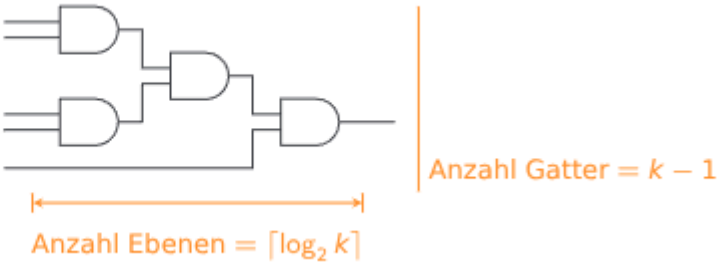
Bemerkung: Technische Realisierung

Eine Boolesche Funktion kann mit **maximal drei Gatterebenen** realisiert werden (Inverter, AND, OR). Sind alle Eingänge sowohl negiert als auch nicht negiert verfügbar, genügen **zwei** Ebenen.

Standardbauteile haben meist nur **zwei Eingänge**. Gatter mit mehr Eingängen werden daher aus mehreren 2-Eingangs-Gattern aufgebaut. Bei k Eingängen:

- Anzahl Gatter: $k - 1$
- Anzahl Ebenen: $\lceil \log_2 k \rceil$

Beispiel für ein Gatter mit $k = 5$ Eingängen aus Standardbauteilen:



---

6. Minimierung

KDNF und KKNF sind zwar eindeutig, aber **nicht optimal** (viele Terme, viele Gatter). Ziel der Minimierung ist ein Ausdruck mit möglichst wenigen Termen und Literalen.

Resolutionsregeln

Unterscheiden sich zwei Summanden/Faktoren in **genau einer** komplementären Variable, können beide Terme durch ihren **gemeinsamen Teil** ersetzt werden.

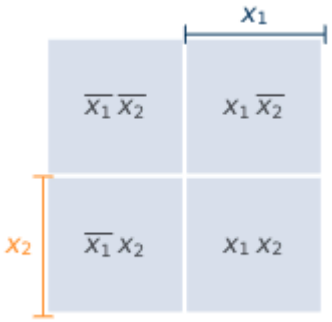
- DNF: $a, x + a, \overline{x} = a$
- KNF: $(a + x)(a + \overline{x}) = a$

Karnaugh-Veitch-Diagramme (KV-Diagramme)

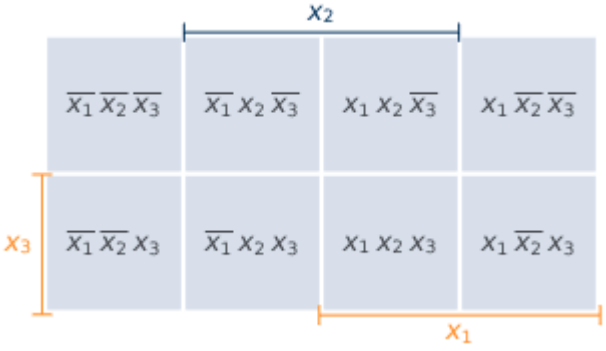
Grafisches Verfahren zur Minimierung. Die Zeilen und Spalten sind so angeordnet (**Gray-Code**), dass sich benachbarte Felder in genau einer Variable unterscheiden. Dadurch lassen sich Resolutionsregeln direkt ablesen: Benachbarte Einsen werden zu Blöcken der Größe 2^k (1, 2, 4, 8, …) zusammengefasst, und jeder Block ergibt einen vereinfachten Term.

Wichtig: Die Ränder des Diagramms gelten als **benachbart** (zyklisch).

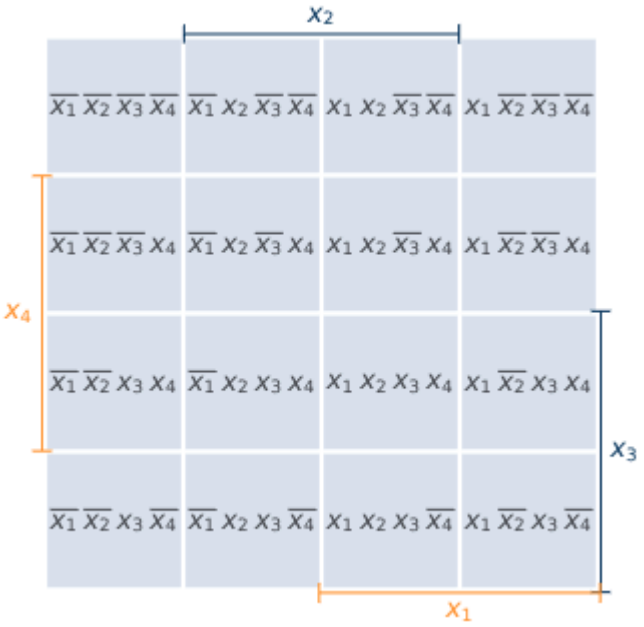
2 Variablen:



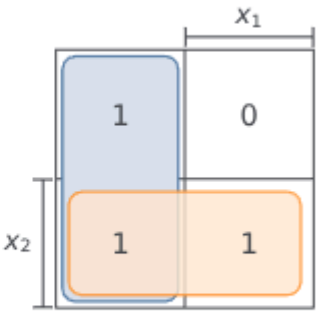
3 Variablen:



4 Variablen:



1. Gegebene KDNF: z. B. $f(x_1, x_2) = \overline{x_1} \cdot \overline{x_2} + \overline{x_1} \cdot x_2 + x_1 \cdot x_2$
2. Erstellen des KV-Diagramms:
   1 für jeden Minterm mit $f(\boldsymbol{x}) = 1$, sonst 0
3. Markierung möglichst weniger, großer, rechteckiger und ggf. überlappender Bereiche aus 2^k Einsen, sodass alle Einsen überdeckt sind
4. Bildung einer minimalen DNF durch Summierung von genau einem Produktterm pro markiertem Bereich: $f(x_1, x_2) = \overline{x_1} + x_2$



7. Typische Schaltnetze

Eine Schaltung kann mehrere Ausgänge haben. Dann gibt es für jeden Ausgang m eine eigene Boolesche Funktion $y_m = f_m(x_1, \ldots, x_n)$.

Dekodierer

Ein **Dekodierer** (Decoder) wandelt einen n-stelligen Binärcode in eine **1-aus-2^n-Darstellung** um: Genau ein Ausgang ist 1, nämlich der, dessen Nummer dem Eingangswert entspricht.

- n Eingänge, 2^n Ausgänge
- Jeder Ausgang ist genau ein **Minterm** der Eingänge.
- Typische Anwendungen: Adressdekodierung (Speicher), Ansteuerung von 7-Segment-Anzeigen, Auswahl von Bausteinen (Chip-Select)

Beispiel: 2-zu-4-Dekodierer

x_1	x_0	y_0	y_1	y_2	y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

$$y_0 = \overline{x_1}, \overline{x_0} \qquad y_1 = \overline{x_1}, x_0 \qquad y_2 = x_1, \overline{x_0} \qquad y_3 = x_1, x_0$$

Oft gibt es zusätzlich einen **Enable-Eingang** E: Ist $E = 0$, sind alle Ausgänge 0. Dann wird jeder Term zusätzlich mit E verundet.

Kodierer

Ein **Kodierer** (Encoder) ist die Umkehrung des Dekodierers: Er wandelt eine **1-aus-2^n-Darstellung** in einen n-stelligen Binärcode um.

- 2^n Eingänge, n Ausgänge
- Es darf **genau ein Eingang** 1 sein. Sind mehrere Eingänge gleichzeitig 1 (oder keiner), ist das Ergebnis undefiniert.

Beispiel: 4-zu-2-Kodierer

x_3	x_2	x_1	x_0	y_1	y_0
0	0	0	1	0	0
0	0	1	0	0	1

x_3	x_2	x_1	x_0	y_1	y_0
0	1	0	0	1	0
1	0	0	0	1	1

$$y_1 = x_2 + x_3 \qquad y_0 = x_1 + x_3$$

(Alle anderen Eingangskombinationen sind nicht zugelassen und werden als Don't Care behandelt, deshalb ist x_0 nicht nötig.)

Prioritätskodierer

Beim **Prioritätskodierer** dürfen mehrere Eingänge gleichzeitig 1 sein. Es zählt der Eingang mit der **höchsten Priorität** (hier: höchster Index). Ein zusätzlicher Ausgang g zeigt an, ob überhaupt ein Eingang aktiv ist.

$$y_1 = x_3 + x_2 \qquad y_0 = x_3 + \overline{x_2}, x_1 \qquad g = x_3 + x_2 + x_1 + x_0$$

Anwendung: Interrupt-Steuerung (welche Unterbrechung wird zuerst bedient?)

Multiplexer

Ein **Multiplexer** (MUX) ist ein **digitaler Umschalter**: Über n **Steuereingänge** (Select) wird einer von 2^n **Dateneingängen** auf den einen Ausgang durchgeschaltet.

- 2^n Dateneingänge d_i, n Steuereingänge s, 1 Ausgang y
- $y = d_i$, wobei i der durch s codierte Wert ist.

2-zu-1-Multiplexer

s	y
0	d_0
1	d_1

$$y = \overline{s}, d_0 + s, d_1$$

4-zu-1-Multiplexer

$$y = \overline{s_1}, \overline{s_0}, d_0 + \overline{s_1}, s_0, d_1 + s_1, \overline{s_0}, d_2 + s_1, s_0, d_3$$

Das ist eine **UND-ODER-Struktur**: Ein Dekodierer der Steuereingänge schaltet das jeweilige UND-Gatter frei, das ODER sammelt alles am Ausgang.

> **Multiplexer als universelle Funktionsgeneratoren** Mit einem 2^n-zu-1-Multiplexer lässt sich **jede** Boolesche Funktion mit n Variablen realisieren: Die Variablen

kommen an die Steuereingänge, an die Dateneingänge werden die Funktionswerte der Wahrheitstabelle (0 oder 1) gelegt.

Demultiplexer

Der **Demultiplexer** (DEMUX) ist die Umkehrung: Ein Dateneingang d wird über die Steuereingänge auf einen von 2^n Ausgängen geleitet, die anderen Ausgänge sind 0.

$$y_i = d \text{ falls } s = i, \text{ sonst } y_i = 0$$

Ein Dekodierer mit Enable-Eingang ist ein Demultiplexer, wenn man den Enable-Eingang als Dateneingang d verwendet.

Bemerkung: OTP Logikanordnung

OTP steht für **One Time Programmable**, also **einmalig programmierbar**. Eine Logikanordnung (z. B. **PLA**/**PAL** oder ein **ROM**) besteht aus zwei Ebenen:

- einer **UND-Ebene**, die Produktterme bildet,
- einer **ODER-Ebene**, die diese zu Summen verknüpft.

Damit lässt sich jede Boolesche Funktion in **disjunktiver Form (DNF)** realisieren. Die Verbindungen zwischen den Leitungen werden einmalig durch das Durchbrennen von Sicherungen (**Fuses**) oder das Herstellen von Verbindungen (**Antifuses**) festgelegt und können danach **nicht mehr verändert** werden.</new_str>